B16 PyTorch hygiene

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B16 PyTorch hygiene 3
B16-L1 Choose the device once, move model and data to it 3
B16-L2 A training loop that does what you think 7
B16-L3 Reproducible PyTorch runs and safe checkpoints 12
Track B best-practice checklist 17
Appendix: solutions and answer keys 18

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B16 PyTorch hygiene

Roadmap: Months 1 to 3: PyTorch foundations. Device handling, training loops and reproducibility.

B16-L1 Choose the device once, move model and data to it

Concept

PyTorch runs on the CPU, on NVIDIA GPUs (`cuda`) and on Apple silicon (`mps`). Every tensor and every model parameter lives on one **device**, and an operation needs all its inputs on the same device. Examples use PyTorch 2.14.

Choose the device once, at the edge, and pass it in:

```python
def pick_device(prefer_gpu: bool = True) -> torch.device:
    if prefer_gpu and torch.cuda.is_available():
        return torch.device("cuda")
    if prefer_gpu and torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")
```

Then move the model once, and each batch as you use it.

The trap: `.to()` **behaves differently on modules and tensors.**

- `model.to(device)` moves the parameters **in place** and returns the same model.
- `tensor.to(device)` returns a **new** tensor. The original stays where it was. `batch.to(device)` on its own line does nothing useful. Write `batch = batch.to(device)`.

Mixing devices raises `RuntimeError`, with a message such as "Tensor on device cpu is not on the expected device". This course's build reproduced it with PyTorch's `meta` device, because no GPU was available.

Other habits:

- Never hardcode `"cuda"`. The same code must run on a laptop CPU, in CI and on a GPU cluster.
- Move tensors to the CPU before converting to NumPy: `preds.cpu().numpy()`. NumPy cannot read GPU memory.
- Log the device with every run (lesson B14-L1). Results can differ slightly between devices.

PyTorch does not run in this app's browser runtime. The exercises check your code's structure, and the device logic uses plain Python.

Why it matters

A training script written on a GPU workstation had `.cuda()` calls on 14 lines. It had to run in CI on CPU machines for a smoke test. Every run failed at import of the first batch. The fix took an afternoon of search and replace, and one `.cuda()` hidden in a helper was missed and broke the nightly job a week later. One `device` chosen at the edge and passed in would have needed no changes.

Worked example: weak code

```
1 model = Net().cuda()
2 for x, y in loader:
2     x.to("cuda")
4     loss = loss_fn(model(x), y.cuda())
```

1. **Line 1.** Hardcoded GPU: fails on any machine without CUDA.
2. **Line 3.** Returns a new tensor that is thrown away. x stays on the CPU.

Worked example: strong code

```
1  from collections.abc import Iterable
2
3  import torch
4
5
1  def pick_device(prefer_gpu: bool = True) -> torch.device:
7      """CUDA, then Apple MPS, then CPU."""
8      if prefer_gpu and torch.cuda.is_available():
9          return torch.device("cuda")
10     if prefer_gpu and torch.backends.mps.is_available():
11         return torch.device("mps")
12     return torch.device("cpu")
13
14
15 def train_epoch(
16     model: torch.nn.Module,
17     loader: Iterable[tuple[torch.Tensor, torch.Tensor]],
18     loss_fn: torch.nn.Module,
19     optimizer: torch.optim.Optimizer,
20     device: torch.device,
21 ) -> float:
22     """One pass over loader. Returns the mean batch loss."""
23     model.train()
24     total, batches = 0.0, 0
25     for x, y in loader:
2          x, y = x.to(device), y.to(device)
```

```
27          optimizer.zero_grad()
28          loss = loss_fn(model(x), y)
29          loss.backward()
30          optimizer.step()
3           total += loss.item()
32          batches += 1
33      return total / batches if batches > 0 else float("nan")
```

1. **Line 6.** One decision, made once, testable on any machine.
2. **Line 26.** Reassigned: the moved tensors are the ones used.
3. **Line 31.** A Python float for logging, not a tensor holding the graph.

Pick the device once and pass it in. Reassign tensors after .to(). Never hardcode "cuda".

Recap card

- **Choose a device** Once, at the edge: cuda if available, then mps, then cpu. Pass it in.
- **Tensor.to vs Module.to** Tensor.to returns a new tensor (reassign). Module.to moves in place.
- **GPU tensor to NumPy** t.detach().cpu().numpy().
- **Hardcoded .cuda()** Breaks on CPU machines and CI. Use .to(device).

Practice

Exercise 1. Multiple choice (level 1)

After `x.to(device)` on its own line, where is `x`?

- a) On device.
- b) Where it was before. .to() on a tensor returns a new tensor.
- c) Copied to both devices.
- d) Deleted.

Why?

- r1) Tensor.to returns a new tensor. Only nn.Module.to changes the object in place.
- r2) .to() only works on models.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.14; the meta device stands in for a GPU)

What you need to know.

`torch.nn.Linear(n_in, n_out)` is a layer with a weight matrix and a bias, both **parameters** the model learns. `model.parameters()` yields them, so

`next(model.parameters())` is the first one. Every tensor has a `.device` saying where it lives, such as `cpu` or `cuda:0`.

```
layer = torch.nn.Linear(3, 2)
next(layer.parameters()).shape    # torch.Size([2, 3])
next(layer.parameters()).device   # device(type='cpu')
```

`torch.ones(2)` is a tensor of two ones.

```
import torch

t = torch.ones(2)
t.to("meta")
model = torch.nn.Linear(2, 1)
model.to("meta")
print(t.device, next(model.parameters()).device)
```

Exercise 3. Fill the gap (level 2)

Move a batch to the chosen device, and bring predictions back for NumPy.

```
x = x._____(1)(device)
preds = model(x).detach()._____(2)().numpy()
```

Exercise 4. Write the function (level 3)

Write `pick_device_name(prefer_gpu: bool, cuda_ok: bool, mps_ok: bool) -> str` that returns `"cuda"`, `"mps"` or `"cpu"`: CUDA first, then MPS, then CPU, and always `"cpu"` when `prefer_gpu` is False. It takes availability as arguments so it runs and tests anywhere. The caller passes `torch.cuda.is_available()` and `torch.backends.mps.is_available()`.

Your code must pass these checks:

- Prefer CUDA, then MPS, then CPU.
- prefer_gpu=False must always give cpu.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Name of the device to use: cuda, then mps, then cpu.
def pick_device_name(
    prefer_gpu: bool,
    cuda_ok: bool,
    mps_ok: bool,
) -> str: ...
```

Starter code:

```
def pick_device_name(prefer_gpu, cuda_ok, mps_ok):
    return "cuda"
```

Exercise 5. Spot the bug (level 4)

Validation runs without errors on the CPU but raises "Expected all tensors to be on the same device" on the GPU. Tap the line that causes it.

What you need to know.

`with torch.inference_mode():` runs the block without tracking gradients. Evaluation needs no gradients, so this saves memory and time. `model.eval()` switches layers such as dropout to evaluation behaviour.

`tensor.argmax(dim=1)` returns, for each row, the position of the largest value: the predicted class.

```
torch.tensor([[0.1, 0.9], [0.8, 0.2]]).argmax(dim=1)   # tensor([1, 0])
```

```
from collections.abc import Iterable

import torch


def validate(model: torch.nn.Module, loader: Iterable[tuple[torch.Tensor,
    torch.Tensor]], device: torch.device) -> float:
    """Accuracy on the validation loader."""
    model.eval()
    correct, total = 0, 0
    with torch.inference_mode():
        for x, y in loader:
            x.to(device)
            preds = model(x).argmax(dim=1)
            correct += int((preds.cpu() == y).sum())
            total += len(y)
    return correct / total if total > 0 else float("nan")
```

Write down the line number of the bug. Then write the fix.

Sources

- In PyTorch 2.14, Tensor.to returns a new tensor and leaves the original unchanged; Module.to moves parameters in place and returns the same module; mixing devices raises RuntimeError. (verified: Run with torch 2.14.0 (CPU) in a scratch environment for this repository, using the meta device.)

B16-L2 A training loop that does what you think

Concept

PyTorch leaves the training loop to you. That gives control, and five ways to get it silently wrong. The standard loop, PyTorch 2.14:

```python
for epoch in range(max_epochs):
    model.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
    val_loss = evaluate(model, val_loader, loss_fn, device)   # model.eval() + torch.inference_mode()
```

The five traps:

1. **Missing `optimizer.zero_grad()`.** Gradients **add up** across `backward()` calls. In this course's build, three backward passes on `2 * w` left a gradient of 6.0 instead of 2.0. Training still runs, with steps that grow each batch.
2. **Missing `model.eval()` for validation.** Dropout stays active and batch norm keeps updating its statistics. Two predictions for the same input differed in train mode and matched in eval mode.
3. **Missing `torch.inference_mode()`** (or `torch.no_grad()`) during validation. PyTorch builds a graph it never uses, wasting memory.
4. **Softmax before `CrossEntropyLoss`.** The loss applies log-softmax itself and expects raw **logits**. For logits `[2.0, 0.5]` and class 0, the correct loss is 0.2014. Passing softmax outputs gave 0.4252, a different loss with weaker gradients.
5. **Summing loss tensors for logging.** `total += loss` keeps every batch's graph alive, and memory grows each step. Use `total += loss.item()`.

Early stopping. Track validation loss each epoch. Stop when it has not improved by at least `min_delta` for `patience` epochs, and keep a copy of the best weights (`copy.deepcopy(model.state_dict())`), not the last ones.

Log the loss curve for both training and validation (lesson B14-L1). A validation loss that rises while training loss falls is overfitting. Both flat from the start usually means a bug, such as frozen parameters or a learning rate of zero.

Why it matters

A text classifier trained for 30 epochs showed a smooth, falling training loss and a validation accuracy stuck at 51%. The code applied `softmax` in the model's `forward` and then used `CrossEntropyLoss`. Gradients were small and the model barely learned. Removing one line lifted validation accuracy to

78% in six epochs. The previous two weeks of hyperparameter tuning had been on a broken loss.

Worked example: weak code

```
for x, y in loader:
    out = torch.softmax(model(x), dim=1)
    loss = nn.CrossEntropyLoss()(out, y)
    loss.backward()
    optimizer.step()
    total += loss
```

1. **Line 2.** CrossEntropyLoss expects logits. Softmax here changes the loss and flattens gradients.
2. **Line 4.** No zero_grad before it: gradients accumulate across batches.
3. **Line 6.** Keeps each batch's graph alive. Use loss.item().

Worked example: strong code

```
def should_stop(val_losses: list[float], patience: int, min_delta: float = 0.0) -> bool:
    """True when the last `patience` epochs did not beat the earlier best by min_delta."""
    if patience < 1:
        raise ValueError("patience must be at least 1")
    if len(val_losses) <= patience:
        return False
    best_before = min(val_losses[:-patience])
    return min(val_losses[-patience:]) > best_before - min_delta
```

1. **Line 5.** Too few epochs to judge: keep training.
2. **Line 8.** Stop only when recent epochs failed to improve on the earlier best.

zero_grad, train and eval modes, inference_mode for validation, logits into CrossEntropyLoss, loss.item() for logs, and early stopping on validation loss.

Recap card

- **The training iteration** to(device), zero_grad, forward and loss, backward, step.
- **Validation mode** model.eval() and torch.inference_mode(). Back to model.train() after.
- **CrossEntropyLoss input** Raw logits. It applies log-softmax itself.
- **Early stopping** Stop when validation loss has not improved by min_delta for patience epochs. Keep the best weights.

Practice

Exercise 1. Multiple choice (level 1)

A loop calls `loss.backward()` and `optimizer.step()` but never `optimizer.zero_grad()`. What happens?

- a) Nothing. PyTorch clears gradients automatically.
- b) Gradients accumulate across batches, so steps use the sum of all earlier gradients.
- c) backward() raises an error on the second batch.
- d) The model stops learning.

Why?

- r1) backward() adds into .grad. Only zero_grad() resets it.
- r2) Optimizers ignore gradients without zero_grad.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.14)

```python
import torch

w = torch.tensor([1.0], requires_grad=True)
for _ in range(3):
    loss = (w * 2).sum()
    loss.backward()
print(w.grad.item())
```

Exercise 3. Put in order (level 2)

Order the steps inside one training iteration.

- optimizer.step()
- loss.backward()
- loss = loss_fn(model(x), y)
- x, y = x.to(device), y.to(device)
- optimizer.zero_grad()

Exercise 4. Write the function (level 3)

Write `should_stop(val_losses: list[float], patience: int, min_delta: float = 0.0) -> bool`. Return `True` when the best loss in the last `patience` epochs is not lower than the best loss before them by more than `min_delta`. Return `False` when there are `patience` or fewer losses. Raise `ValueError` if `patience < 1`.

Your code must pass these checks:

- No improvement on 0.9 in the last 2 epochs: stop.
- An improvement in the window, or too few epochs, means continue.
- Improvements smaller than min_delta do not count.
- patience below 1 must raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Early-stopping rule on validation losses, one per epoch.
def should_stop(
    val_losses: list[float],
    patience: int,
    min_delta: float = 0.0,
) -> bool: ...
```

Starter code:

```python
def should_stop(val_losses, patience, min_delta=0.0):
    return val_losses[-1] > val_losses[-2]
```

Exercise 5. Spot the bug (level 4)

Validation accuracy is stuck near chance while training loss falls slowly. Tap the line that breaks the loss.

What you need to know.

`nn.Sequential(a, b, c)` chains layers: the output of one is the input of the next. `nn.Linear(n_in, n_out)` is a fully connected layer, and `nn.ReLU()` sets negative values to 0. The class stores the chain as `self.net`, and `forward` decides what the model returns.

```python
net = nn.Sequential(nn.Linear(3, 4), nn.ReLU(), nn.Linear(4, 2))
net(torch.ones(1, 3)).shape   # torch.Size([1, 2]): one score per class
```

```python
import torch
from torch import nn


class Classifier(nn.Module):
    """Two-layer classifier. Train with nn.CrossEntropyLoss."""

    def __init__(self, n_in: int, n_classes: int) -> None:
        super().__init__()
        self.net = nn.Sequential(nn.Linear(n_in, 32), nn.ReLU(),
    nn.Linear(32, n_classes))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.softmax(self.net(x), dim=1)
```

Write down the line number of the bug. Then write the fix.

Sources

- Three backward passes on 2*w leave w.grad = 6.0; dropout outputs differ in train mode and match in eval mode; CrossEntropyLoss on logits [2.0, 0.5] with target 0 is 0.2014, and 0.4252 on their softmax. (verified: Run with torch 2.14.0 (CPU) in a scratch environment for this repository.)

B16-L3 Reproducible PyTorch runs and safe checkpoints

Concept

Neural network training draws random numbers everywhere: weight initialisation, data shuffling, dropout, augmentation. Control every source and two runs with the same seed give the same weights. In this course's build (PyTorch 2.14, CPU), a small network trained twice with seed 1 produced identical parameters. Seed 2 produced different ones.

Seed at the entry point, once. Your own code takes generators as parameters (lesson B6-L1). Libraries and PyTorch internals use global generators, so seed those too, at the start of the training script:

```
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)                          # CPU and all CUDA devices
loader_gen = torch.Generator().manual_seed(seed)
train_loader = DataLoader(ds, batch_size=64, shuffle=True,
    generator=loader_gen)
```

- Pass a seeded `generator` to every shuffling `DataLoader`, so batch order is fixed.
- With `num_workers > 0`, the PyTorch documentation recommends a `worker_init_fn` that seeds NumPy and `random` in each worker from `torch.initial_seed()`.
- `torch.use_deterministic_algorithms(True)` makes PyTorch raise an error for operations that have no deterministic implementation, instead of silently varying. On CUDA it also needs the `CUBLAS_WORKSPACE_CONFIG` environment variable, as the documentation describes.

Exact equality holds for the same code, library versions and hardware. Across GPU models or library versions, expect small differences, and compare metrics within a tolerance (lesson B14-L3).

Checkpoints let a long run resume and let you keep the best epoch. Save a dict of state, not the model object:

```
torch.save({"model": model.state_dict(), "optimizer":
    optimizer.state_dict(),
            "epoch": epoch, "torch_rng": torch.get_rng_state()}, path)
```

`torch.load` loads **weights only** by default in PyTorch 2.14. A checkpoint holding a pickled model object fails with `UnpicklingError`. That default protects you: loading an arbitrary pickle can execute code. Save state dicts, rebuild the model class in code, and load the weights into it.

Why it matters

A team fine-tuned a classifier and reported 0.86 F1. A reviewer reran the notebook and got 0.83, then 0.85. The DataLoader shuffled with the global generator, dropout drew from it too, and an augmentation library used its own. Nobody could say which number was "the" result. Fixing the seeds and the loader generator made the run repeatable: 0.84, every time. The earlier 0.86 had been a lucky draw.

Worked example: weak code

```
model = torch.load("best_model.pt")
loader = DataLoader(ds, batch_size=64, shuffle=True)
```

1. **Line 1.** Expects a pickled model object: refused by the weights-only default, and unsafe if forced.
2. **Line 2.** Shuffles with the global generator: batch order changes with any other random call.

Worked example: strong code

```
import random
from pathlib import Path

import numpy as np
import torch


def seed_everything(seed: int) -> torch.Generator:

    """Seed Python, NumPy and PyTorch global state; return a generator for
    DataLoaders."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    return torch.Generator().manual_seed(seed)


def save_checkpoint(path: Path, model: torch.nn.Module, optimizer:
    torch.optim.Optimizer, epoch: int) -> None:
```

```
    """Save state dicts and RNG state, never the model object."""
    torch.save(
        {
            "model": model.state_dict(),
            "optimizer": optimizer.state_dict(),
            "epoch": epoch,
            "torch_rng": torch.get_rng_state(),
        },
        path,
    )
```

1. **Line 13.** A dedicated generator for DataLoader shuffling.
2. **Line 20.** Weights as tensors: loads under the weights-only default.
3. **Line 23.** Resuming restores the random stream as well as the weights.

Seed every generator at the entry point, give DataLoaders their own generator, and checkpoint state dicts, not objects.

Recap card

- **Seed a PyTorch run** random.seed, np.random.seed, torch.manual_seed at the entry point, plus a seeded Generator for each DataLoader.
- **torch.load default in PyTorch 2.14** Weights only. Pickled objects fail with UnpicklingError.
- **Checkpoint contents** Model and optimizer state dicts, epoch, RNG state, best metric.
- **Deterministic operations** torch.use_deterministic_algorithms(True) raises on operations with no deterministic version.

Practice

Exercise 1. Multiple choice (level 1)

`torch.load("model.pt")` raises `UnpicklingError` in PyTorch 2.14. The file was saved with `torch.save(model, path)`. Best fix?

- a) Always pass weights_only=False.
- b) Save model.state_dict() instead, rebuild the model class in code, and load the weights into it.
- c) Downgrade PyTorch.
- d) Rename the file to .pkl.

Why?

- r1) State dicts hold only tensors and plain values, which load safely under the weights-only default.
- r2) Model objects cannot be saved.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.14)

What you need to know.

`torch.randperm(n, generator=g)` returns the numbers 0 to n - 1 in random order, drawn from generator `g`. Without `generator=`, random functions such as `torch.rand(100)` draw from PyTorch's global generator.

```
g = torch.Generator().manual_seed(1)
torch.randperm(4, generator=g).tolist()   # [1, 3, 2, 0]
```

```
import torch

g1 = torch.Generator().manual_seed(0)
g2 = torch.Generator().manual_seed(0)
a = torch.randperm(5, generator=g1).tolist()
torch.rand(100)
b = torch.randperm(5, generator=g2).tolist()
print(a == b)
```

Exercise 3. Fill the gap (level 2)

Give a shuffling DataLoader its own seeded generator.

```
loader_gen = torch.Generator()._____(1)(seed)
loader = DataLoader(ds, batch_size=64, shuffle=True, _____(2)=loader_gen)
```

Exercise 4. Write the function (level 3)

Write `seed_everything(seed: int) -> torch.Generator`. Seed Python's `random`, NumPy's global generator and PyTorch with `torch.manual_seed`, then return `torch.Generator().manual_seed(seed)` for DataLoaders.

PyTorch does not run in the browser. The checker reads your code.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Seed Python, NumPy and PyTorch global state at the entry point.
def seed_everything(seed: int) -> torch.Generator: ...
```

Starter code:

```
import torch


def seed_everything(seed):
    torch.manual_seed(seed)
```

Exercise 5. Architecture design (level 4)

Design checkpointing for a 12-hour training job that must resume after interruptions and keep the best epoch.

Options: Model and optimizer state dicts, epoch, RNG state, best validation loss, The whole model object with torch.save(model).

1. Checkpoint content
2. When to save
3. Loading

Sources

- Same-seed training on CPU gave identical parameters; torch.load of a pickled model raised UnpicklingError; a dict of state dicts, epoch and torch.get_rng_state() loaded with the default. (verified: Run with torch 2.14.0 (CPU) in a scratch environment for this repository.)
- PyTorch recommends a worker_init_fn seeding NumPy and random from torch.initial_seed(), and CUBLAS_WORKSPACE_CONFIG for deterministic CUDA. (search-only: PyTorch documentation, Reproducibility notes (not reachable from this environment).)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B16-L1 Choose the device once, move model and data to it

B16-L1 exercise 1

Hints. (1) Is Tensor.to in place? (2) What happens to an unassigned return value? (3) Reassign.

Answer. b) Where it was before. .to() on a tensor returns a new tensor. Because: r1) Tensor.to returns a new tensor. Only nn.Module.to changes the object in place.

`x = x.to(device)`. For modules, `model.to(device)` is in place and returns the same model.

Common wrong answers.

- `a`: That is true for models, not tensors. Write x = x.to(device).
- `c`: The new tensor is discarded because it is not assigned.
- `d`: x is unchanged. The moved copy is what disappears.
- `b|r2`: Right answer, wrong reason. .to() works on tensors, it just does not modify them.

B16-L1 exercise 2

Hints. (1) Which call is in place? (2) Module.to. (3) Tensor.to returns a copy.

Answer.

```
1 cpu meta
```

The asymmetry catches everyone once. Reassign tensors, and do not rely on reassigning models.

Common wrong answers.

- `meta meta`: Tensor.to returns a new tensor. t itself stays on the CPU.
- `cpu cpu`: Module.to moves parameters in place, so the model is on meta.

B16-L1 exercise 3

Hints. (1) The device-agnostic method. (2) NumPy needs CPU memory. (3) `to` and `cpu`.

Answer.

```
x = x.to(device)
preds = model(x).detach().cpu().numpy()
```

`.to(device)` keeps code portable. `.cpu()` before `.numpy()` avoids an error on GPU tensors.

Common wrong answers.

- g1 = `cuda`: .cuda() hardcodes the GPU. .to(device) works on any machine.

B16-L1 exercise 4

Hints. (1) Check prefer_gpu in every GPU branch. (2) CUDA before MPS. (3) Fall through to cpu.

Answer.

```
def pick_device_name(prefer_gpu: bool, cuda_ok: bool, mps_ok: bool) ->
    str:
    """Name of the device to use: cuda, then mps, then cpu.

    Args:
        prefer_gpu: False forces the CPU, for tests and CI.
        cuda_ok: Result of torch.cuda.is_available().
        mps_ok: Result of torch.backends.mps.is_available().

    Returns:
        "cuda", "mps" or "cpu".
    """
    if prefer_gpu and cuda_ok:
        return "cuda"
    if prefer_gpu and mps_ok:
        return "mps"
    return "cpu"
```

Passing availability in makes the decision a pure function (lesson B2-L1): easy to test on a laptop with no GPU.

B16-L1 exercise 5

Hints. (1) Which line tries to move x? (2) Does Tensor.to change x? (3) Reassign it.

Answer. The bug is on line 12.

Fixed version:

```
from collections.abc import Iterable

import torch


```

```
 6 def validate(model: torch.nn.Module, loader: Iterable[tuple[torch.Tensor,
     torch.Tensor]], device: torch.device) -> float:
 7     """Accuracy on the validation loader."""
 8     model.eval()
 9     correct, total = 0, 0
10     with torch.inference_mode():
11         for x, y in loader:
12             x = x.to(device)
13             preds = model(x).argmax(dim=1)
14             correct += int((preds.cpu() == y).sum())
15             total += len(y)
16     return correct / total if total > 0 else float("nan")
```

On the CPU, x and the model already share a device, so the bug hides. On the GPU, x never moved. Reassign: `x = x.to(device)`.

Common wrong answers.

- Line 14: preds.cpu() is right: y stays on the CPU, so the comparison happens there.
- Line 8: model.eval() is correct for validation.

B16-L2 A training loop that does what you think

B16-L2 exercise 1

Hints. (1) Does backward overwrite or add? (2) It adds. (3) zero_grad resets.

Answer. b) Gradients accumulate across batches, so steps use the sum of all earlier gradients. Because: r1) backward() adds into .grad. Only zero_grad() resets it.

Call `optimizer.zero_grad()` before each backward pass.

Common wrong answers.

- `a`: PyTorch accumulates on purpose, for gradient accumulation. Clearing is your job.
- `c`: No error: training runs with growing, wrong updates.
- `d`: It keeps changing, with updates that grow each batch.
- `b|r2`: Right answer, wrong reason. The optimizer uses the accumulated gradients.

B16-L2 exercise 2

Hints. (1) The gradient of 2w is 2. (2) How many backward calls? (3) They add up.

Answer.

```
6.0
```

Three accumulations of 2.0. With `zero_grad()` each iteration it would print 2.0.

Common wrong answers.

- `2.0`: Each backward adds 2.0 to w.grad. Three calls without zeroing give 6.0.

B16-L2 exercise 3

Hints. (1) Data first. (2) Clear gradients before computing new ones. (3) Compute, then apply.

Answer.

1. x, y = x.to(device), y.to(device)
2. optimizer.zero_grad()
3. loss = loss_fn(model(x), y)
4. loss.backward()
5. optimizer.step()

Move, clear, forward, backward, step. zero_grad can also sit at the end of the iteration, as long as it runs between steps.

Common wrong answers.

- zero_grad after backward erases the gradients the step needs.
- step before backward uses stale or empty gradients.

B16-L2 exercise 4

Hints. (1) Split the list: before the window and the window. (2) Compare the window's best with the earlier best. (3) Subtract min_delta from the earlier best.

Answer.

```python
def should_stop(val_losses: list[float], patience: int, min_delta: float = 0.0) -> bool:
    """Early-stopping rule on validation losses, one per epoch.

    Args:
        val_losses: Validation loss per epoch, oldest first.
        patience: Epochs allowed without improvement.
        min_delta: Smallest decrease that counts as improvement.

    Returns:
        True when training should stop.

    Raises:
```

```
        ValueError: If patience is below 1.
    """
    if patience < 1:
        raise ValueError("patience must be at least 1")
    if len(val_losses) <= patience:
        return False
    best_before = min(val_losses[:-patience])
    return min(val_losses[-patience:]) > best_before - min_delta
```

Comparing best against best ignores noisy single epochs. min_delta stops training from continuing on improvements too small to matter.

B16-L2 exercise 5

Hints. (1) What does CrossEntropyLoss expect as input? (2) What does forward return? (3) Logits, not probabilities.

Answer. The bug is on line 13.

Fixed version:

```
import torch
from torch import nn


class Classifier(nn.Module):
    """Two-layer classifier. Train with nn.CrossEntropyLoss."""

    def __init__(self, n_in: int, n_classes: int) -> None:
        super().__init__()
        self.net = nn.Sequential(nn.Linear(n_in, 32), nn.ReLU(),
    nn.Linear(32, n_classes))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

Softmax squeezes outputs into [0, 1], and CrossEntropyLoss then applies log-softmax again. The loss and its gradients are wrong. Apply softmax only when you need probabilities for reporting.

Common wrong answers.

- Line 10: The architecture is fine. The output passed to the loss is the problem.

B16-L3 Reproducible PyTorch runs and safe checkpoints

B16-L3 exercise 1

Hints. (1) What does weights-only loading allow? (2) What does a state dict contain? (3) Tensors and plain values.

Answer. b) Save model.state_dict() instead, rebuild the model class in code, and load the weights into it. Because: r1) State dicts hold only tensors and plain values, which load safely under the weights-only default.

Save `state_dict()`, keep the model class in version-controlled code, and load weights into a fresh instance.

Common wrong answers.

- `a`: That loads arbitrary pickles, which can run code. Only do it for files you created and trust, and prefer fixing how you save.
- `c`: Downgrading removes a safety default and freezes you on an old version.
- `d`: The extension changes nothing about the content.
- `b|r2`: Right answer, wrong reason. Objects can be saved. Loading them safely is the problem.

B16-L3 exercise 2

Hints. (1) Which generator does torch.rand use here? (2) Do g1 and g2 share state with it? (3) No.

Answer.

```
1 True
```

Dedicated generators isolate the shuffle order from other random calls. That is why DataLoaders get their own.

Common wrong answers.

- `False`: torch.rand(100) uses the global generator. g1 and g2 are separate and unaffected.

B16-L3 exercise 3

Hints. (1) The Generator method that seeds it. (2) The DataLoader argument that takes it. (3) `manual_seed` and `generator`.

Answer.

```
1 loader_gen = torch.Generator().manual_seed(seed)
2 loader = DataLoader(ds, batch_size=64, shuffle=True, generator=loader_gen)
```

`torch.Generator().manual_seed(seed)` returns the generator, ready to pass as `generator=`.

Common wrong answers.

- g2 = `seed`: DataLoader has no seed argument. Pass a torch.Generator as generator=.

B16-L3 exercise 4

Hints. (1) Three global generators: random, NumPy, torch. (2) One dedicated generator for the loader. (3) `return torch.Generator().manual_seed(seed)`.

Answer.

```python
import random

import numpy as np
import torch


def seed_everything(seed: int) -> torch.Generator:
    """Seed Python, NumPy and PyTorch global state at the entry point.

    Args:
        seed: The run's seed, logged with the run.

    Returns:
        A seeded generator to pass to shuffling DataLoaders.
    """
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    return torch.Generator().manual_seed(seed)
```

Seeding every global generator at the entry point covers code you do not control. The returned generator keeps batch order independent of everything else.

B16-L3 exercise 5

Hints. (1) What must be restored to continue exactly? (2) Which two checkpoints serve different needs? (3) Keep the safe default.

Answer.

- State dicts plus epoch, RNG state and best score: everything needed to resume exactly.
- Save the last epoch for resuming, and the best epoch for release.
- Load with the safe default into a model defined in code.

Resume from state, release the best epoch, and never unpickle files you did not create.

Common wrong answers.

- content = `object`: A pickled object ties the file to the exact code layout and fails the weights-only default.

- when = `end`: An interruption at hour 11 loses everything, and the last epoch is rarely the best.
- load = `unsafe`: Unpickling files from shared storage can run arbitrary code.